

Chapter 4 – Sequential explicit convex approximations



4.1 General solution procedure for nested problems

- Formulation of a nested structural optimization problem (SO_{nf}):

$$\begin{aligned} \min_{\mathbf{x}} : \quad & g_0(\mathbf{x}, \mathbf{u}(\mathbf{x})) \\ \text{s.t.} : \quad & g_i(\mathbf{x}, \mathbf{u}(\mathbf{x})) \leq 0, \quad i = 1, \dots, l \\ & \mathbf{x} \in \mathcal{X} \end{aligned}$$

$$\text{where } \mathbf{u}(\mathbf{x}) = \mathbf{K}^{-1}(\mathbf{x})\mathbf{F}(\mathbf{x})$$

or:

$$\begin{aligned} \min_{\mathbf{x}} : \quad & \hat{g}_0(\mathbf{x}) \\ \text{s.t.} : \quad & \hat{g}_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, l \\ & \mathbf{x} \in \mathcal{X} \end{aligned}$$

$$\text{where } \hat{g}_i(\mathbf{x}) = g_i(\mathbf{x}, \mathbf{u}(\mathbf{x})), \quad i = 0, \dots, l$$

- There is no explicit expression available for $\mathbf{u}(\mathbf{x})$ or $\hat{g}_i(\mathbf{x})$, but for every value of $\mathbf{x}$ we can compute the values and derivatives of $\mathbf{u}$ and $\hat{g}_i$.

4.1 General solution procedure for nested problems

- The problem ($\mathbb{SO}_{\text{nf}}$) will be solved by generating and solving a sequence of explicit subproblems that are approximations of ($\mathbb{SO}_{\text{nf}}$).
- The subproblems are formulated taking into account the function values $\hat{g}_i$ and possibly their derivatives.
- If the derivatives up to order j are used, the optimization algorithm is said to be of order j . In structural optimization, first-order algorithms are most common.
- Procedure:
 - (1) Start with an initial design $\mathbf{x}^0$. Set the iteration counter $k = 0$.
 - (2) Calculate the displacement vector $\mathbf{u}(\mathbf{x}^k)$ for the current design.
 - (3) For the current design, calculate the objective function $\hat{g}_0(\mathbf{x}^k)$, the constraint functions $\hat{g}_i(\mathbf{x}^k)$, $i = 1, \dots, l$, and their gradients $\nabla \hat{g}_i(\mathbf{x}^k)$, $i = 0, \dots, l$.
 - (4) Formulate an explicit convex approximation ($\mathbb{SO}_{\text{nf}}^k$) at $\mathbf{x}^k$ of ($\mathbb{SO}_{\text{nf}}$).
 - (5) Solve ($\mathbb{SO}_{\text{nf}}^k$) by a convex optimization algorithm to give a new design $\mathbf{x}^{k+1}$.
 - (6) Put $k = k + 1$ and return to step 2 unless a stopping criterion is satisfied.
- From now on the function $\hat{g}_i$ is denoted as g_i .

4.2 Sequential linear programming (SLP)

- The objective function and the constraints are linearized at $\mathbf{x}^k$ (they are replaced with their first-order Taylor expansion), leading to the following explicit convex subproblem (SO_{nf}^k):

$$\begin{aligned} \min_{\mathbf{x}} : \quad & g_0(\mathbf{x}^k) + \nabla g_0(\mathbf{x}^k)^T (\mathbf{x} - \mathbf{x}^k) \\ \text{s.t.} : \quad & g_i(\mathbf{x}^k) + \nabla g_i(\mathbf{x}^k)^T (\mathbf{x} - \mathbf{x}^k) \leq 0, \quad i = 1, \dots, l \\ & \mathbf{x} \in \mathcal{X} \\ & \alpha_j^k \leq x_j \leq \beta_j^k, \quad j = 1, \dots, n \end{aligned}$$

- The move limits α_j^k and β_j^k must be updated adaptively.
- The subproblem (SO_{nf}^k) is a linear program and can be solved by e.g. the simplex method.

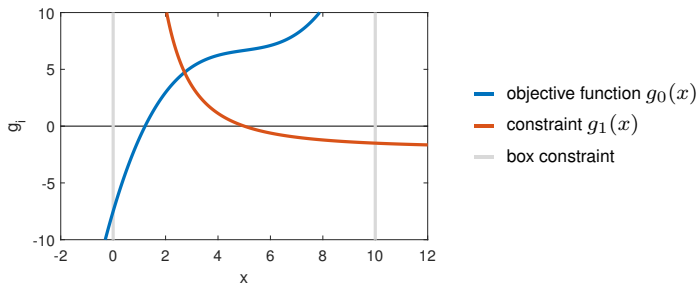
4.2 Sequential linear programming (SLP)

Example: optimization problem with a single design variable and a single constraint.

$$\min_x : g_0(x) = \frac{1}{3}x + \frac{1}{10}(x-5)^3 + 5$$

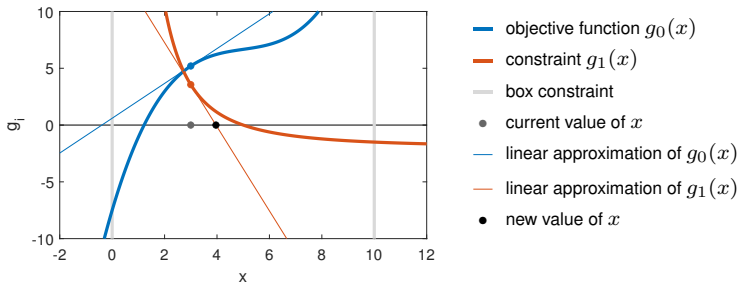
$$\text{s.t.} : g_1(x) = \frac{50}{x^2} - 2 \leq 0$$

$$0 \leq x \leq 10$$



4.2 Sequential linear programming (SLP)

Example: optimization problem with a single design variable and a single constraint.



- Solve the problem in MATLAB with starting point $x = 3$ (ex1a.m, case 1).
- Change the starting point to $x = 10$ (ex1a.m, case 2).
- Increase the lower box constraint to $x_{\min} = 0.1$ (ex1a.m, case 3).
- Add a move limit of 0.2 (ex1a.m, case 4).
- Tighten the move limit to 0.02 (ex1a.m, case 5).

4.3 Sequential quadratic programming (SQP)

- A second-order term is added to the Taylor expansion of the objective function:

$$\min_{\mathbf{x}} : g_0(\mathbf{x}^k) + \nabla g_0(\mathbf{x}^k)^T(\mathbf{x} - \mathbf{x}^k) + \frac{1}{2}(\mathbf{x} - \mathbf{x}^k)\mathbf{H}(\mathbf{x}^k)(\mathbf{x} - \mathbf{x}^k)$$

$$\text{s.t.} : g_i(\mathbf{x}^k) + \nabla g_i(\mathbf{x}^k)^T(\mathbf{x} - \mathbf{x}^k) \leq 0, \quad i = 1, \dots, l$$

$$\mathbf{x} \in \mathcal{X}$$

- The move limits are omitted as they are far less important than for SLP.
- The matrix $\mathbf{H}$ is either the actual Hessian of the objective function or a first-order positive definite approximation.

4.4 Convex linearization (CONLIN)

- The objective function and the constraints are expressed as linear combinations of either x_j (for increasing functions) or $1/x_j$ (for decreasing functions):

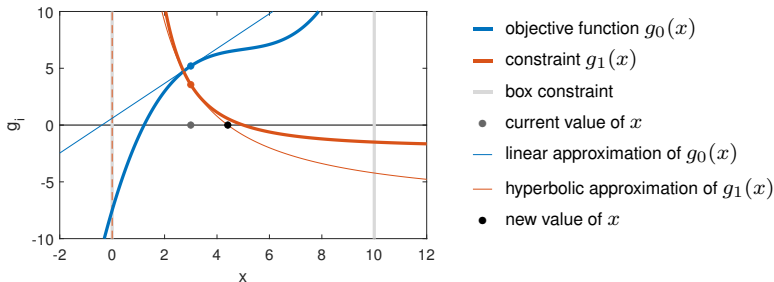
$$\begin{aligned} \min_{\mathbf{x}} : \quad & r_0^k + \sum_{j=1}^n \left(p_{0j}^k x_j + \frac{q_{0j}^k}{x_j} \right) \\ \text{s.t.} : \quad & r_i^k + \sum_{j=1}^n \left(p_{ij}^k x_j + \frac{q_{ij}^k}{x_j} \right) \leq 0, \quad i = 1, \dots, l \\ & \mathbf{x} \in \mathcal{X} \\ & \alpha_j^k \leq x_j \leq \beta_j^k, \quad j = 1, \dots, n \end{aligned}$$

where:

$$\begin{aligned} p_{ij}^k &= \begin{cases} \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} & \text{if } \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} > 0 \\ 0 & \text{otherwise} \end{cases} \\ q_{ij}^k &= \begin{cases} -(x_j^k)^2 \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} & \text{if } \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} < 0 \\ 0 & \text{otherwise} \end{cases} \\ r_i^k &= g_i(\mathbf{x}^k) - \sum_{j=1}^n \left(p_{ij}^k x_j^k + \frac{q_{ij}^k}{x_j^k} \right) \end{aligned}$$

4.4 Convex linearization (CONLIN)

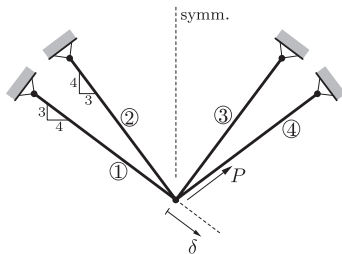
Example: optimization problem with a single design variable and a single constraint.



- Solve the problem in MATLAB with starting point $x = 3$ (ex1a.m, case 6).
- Change the starting point to $x = 10$ (ex1a.m, case 7).

4.4 Convex linearization (CONLIN)

Example: weight minimization of a four-bar truss subject to a displacement constraint.



Bar length l , Young's modulus E , section areas $A_3 = A_2$ and $A_4 = A_1$

Maximum elongation of bar 1: $\delta \leq \delta_0$

Reference area $A_0 = PL/(10\delta_0 E)$

Minimum section area $0.2A_0$, maximum section area $2.5A_0$

Design variables $x_i = A_i/A_0$

– Solve the problem in MATLAB (ex2a.m).

4.5 Method of moving asymptotes (MMA)

- The objective function and the constraints are expressed as linear combinations of either increasing or decreasing hyperbolic functions $1/(U_j - x_j)$ or $1/(x_j - L_j)$:

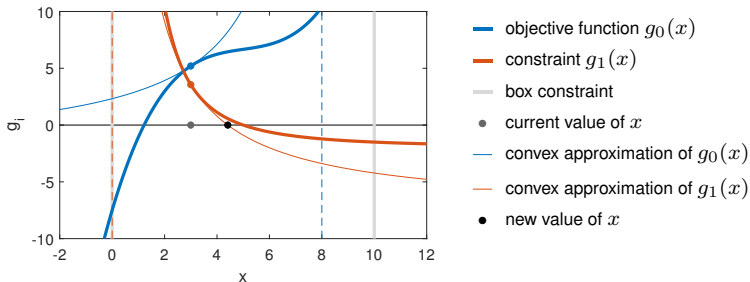
$$\begin{aligned} \min_{\mathbf{x}} : \quad & r_0^k + \sum_{j=1}^n \left(\frac{p_{0j}^k}{U_j^k - x_j} + \frac{q_{0j}^k}{x_j - L_j^k} \right) \\ \text{s.t.} : \quad & r_i^k + \sum_{j=1}^n \left(\frac{p_{ij}^k}{U_j^k - x_j} + \frac{q_{ij}^k}{x_j - L_j^k} \right) \leq 0, \quad i = 1, \dots, l \\ & \mathbf{x} \in \mathcal{X} \\ & \alpha_j^k \leq x_j \leq \beta_j^k, \quad j = 1, \dots, n \end{aligned}$$

where:

$$\begin{aligned} p_{ij}^k &= \begin{cases} (U_j^k - x_j^k)^2 \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} & \text{if } \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} > 0 \\ 0 & \text{otherwise} \end{cases} \\ q_{ij}^k &= \begin{cases} -(x_j^k - L_j^k)^2 \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} & \text{if } \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} < 0 \\ 0 & \text{otherwise} \end{cases} \\ r_i^k &= g_i(\mathbf{x}^k) - \sum_{j=1}^n \left(\frac{p_{ij}^k}{U_j^k - x_j^k} + \frac{q_{ij}^k}{x_j^k - L_j^k} \right) \end{aligned}$$

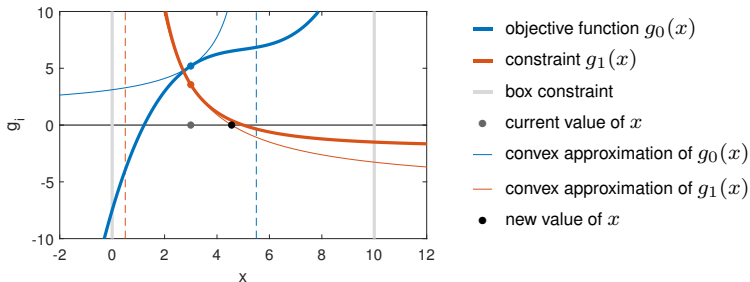
4.5 Method of moving asymptotes (MMA)

Example: optimization problem with a single design variable and a single constraint.



4.5 Method of moving asymptotes (MMA)

Same example with tighter asymptotes.



4.5 Method of moving asymptotes (MMA)

Heuristic updating scheme for the asymptotes L_j^k and U_j^k and the move limits α_j^k and β_j^k :

- For $k = 0$ and $k = 1$:

$$L_j^k = x_j^k - s_{\text{init}}(x_j^{\text{max}} - x_j^{\text{min}})$$

$$U_j^k = x_j^k + s_{\text{init}}(x_j^{\text{min}} - x_j^{\text{max}})$$

- For $k \geq 2$, if $(x_j^k - x_j^{k-1})(x_j^{k-1} - x_j^{k-2}) > 0$:

$$L_j^k = x_j^k - s_{\text{incr}}(x_j^{k-1} - L_j^{k-1})$$

$$U_j^k = x_j^k + s_{\text{incr}}(U_j^{k-1} - x_j^{k-1})$$

- For $k \geq 2$, if $(x_j^k - x_j^{k-1})(x_j^{k-1} - x_j^{k-2}) < 0$:

$$L_j^k = x_j^k - s_{\text{decr}}(x_j^{k-1} - L_j^{k-1})$$

$$U_j^k = x_j^k + s_{\text{decr}}(U_j^{k-1} - x_j^{k-1})$$

- Move limits:

$$\alpha_j^k = \max(x_j^{\text{min}}, L_j^k + \mu(x_j^k - L_j^k))$$

$$\beta_j^k = \min(x_j^{\text{max}}, U_j^k - \mu(U_j^k - x_j^k))$$

Method of moving asymptotes (MMA) – updated version

- The current version of MMA, developed in 2007/2008, uses a slightly different definition for the coefficients p_{ij}^k and q_{ij}^k of the hyperbolic functions:

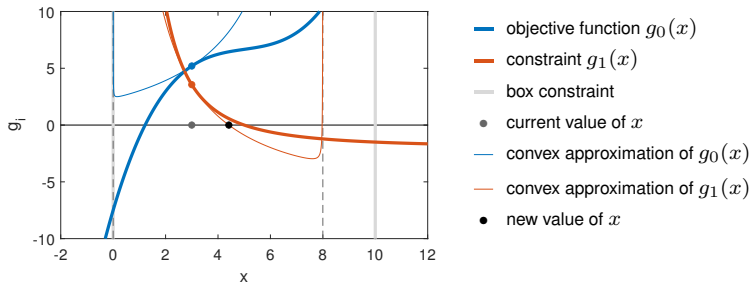
$$p_{ij}^k = (U_j^k - x_j^k)^2 \left(1.001 \left(\frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} \right)^+ + 0.001 \left(\frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} \right)^- + \frac{\rho_i}{x_j^{\max} - x_j^{\min}} \right)$$
$$q_{ij}^k = (x_j^k - L_j^k)^2 \left(0.001 \left(\frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} \right)^+ + 1.001 \left(\frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} \right)^- + \frac{\rho_i}{x_j^{\max} - x_j^{\min}} \right)$$

where $(\diamond)^+$ is defined as $\max(0, \diamond)$, $(\diamond)^-$ is defined as $\max(0, -\diamond)$, and $\rho_i = 10^{-5}$ is a low positive number.

This version of MMA can be downloaded from `smoptit.se`. See the included PDF file for more information. The function `mma` in the `structopt` toolbox is based on this version.

Method of moving asymptotes (MMA) – updated version

Example: optimization problem with a single design variable and a single constraint.



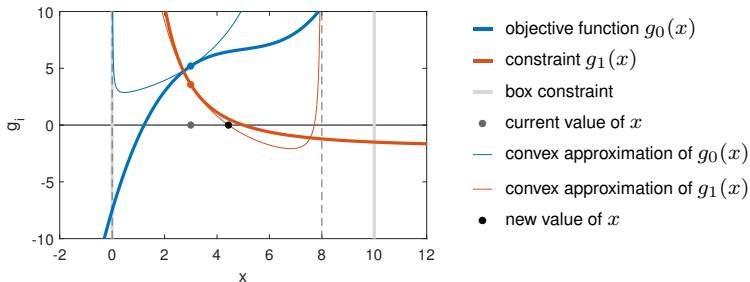
- Solve the problem in MATLAB with starting point $x = 3$, $s_{\text{init}} = 0.2$, $s_{\text{incr}} = 1.2$, $s_{\text{decr}} = 0.7$, and $\mu = 0$ (ex1a.m, case 8).
- Start with tighter asymptotes using $s_{\text{init}} = 0.02$ (ex1a.m, case 9).
- Let the asymptotes approximately go to $-\infty$ and ∞ (ex1a.m, case 10).
- Let the asymptotes approximately go to 0 and ∞ (ex1a.m, case 11).

Globally convergent method of moving asymptotes (GCMMA)

- The globally convergent version of MMA verifies if the convex approximations are conservative, i.e. if they are larger than or equal to the actual values of $g_i(\mathbf{x})$ in the new iteration point.
- If this is not the case, the new iteration point is discarded, and the parameters ρ_i are increased to make the approximations more conservative.
- Due to the conservative nature of the convex approximations in GCMMA, each iteration leads to a feasible point with a lower objective value than the previous one.
- The sequence of iteration points converges towards one of the KKT points (but not necessarily to the global optimum).
- An implementation of GCMMA can be downloaded from `smoptit.se`. See the included PDF file for more information. The function `gcmma` in the `structopt` toolbox is based on this implementation.

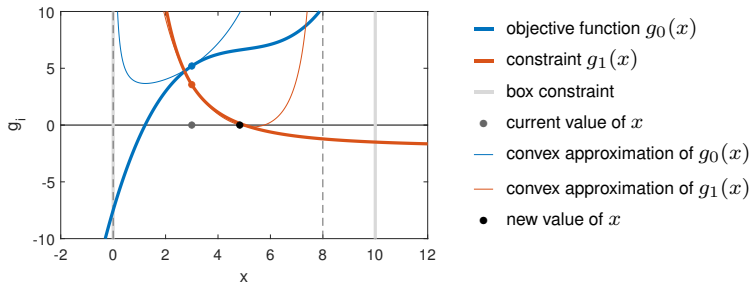
Globally convergent method of moving asymptotes (GCMMA)

Example: optimization problem with a single design variable and a single constraint.



Globally convergent method of moving asymptotes (GCMMA)

Same example with a more conservative approximation (higher value of ρ_i):



- Solve the problem in MATLAB with starting point $x = 3$, $s_{\text{init}} = 0.5$, $s_{\text{incr}} = 1.2$, $s_{\text{decr}} = 0.7$, and $\mu = 0$ (ex1b.m).

MASTER OF ENGINEERING: ARCHITECTURE - CIVIL ENGINEERING

H0T88A Structural Optimization

Exercise 4.6

Source: P.W. Christensen and A. Klarbring. An introduction to structural optimization. Springer, 2008.

In this exercise we use MATLAB to solve some simple sizing optimization problems using MMA. The aim of the exercise is to get a better understanding of how MMA works by visualizing the approximated subproblems generated by the algorithm. The code is available on Toledo.

Consider the problem of minimizing the weight of a beam consisting of two segments, cfr. section 2.4. The problem may be written as a mathematical programming problem as:

$$(\mathbb{P})_1 \quad \begin{cases} \min_{x_1, x_2} & g_0(x_1, x_2) = x_1 + x_2 \\ \text{s.t.} & \begin{cases} g_1(x_1, x_2) = \frac{1}{x_1^3} + \frac{7}{x_2^3} - 1 \leq 0 \\ 0.1 \leq x_1 \leq 10, \quad 0.1 \leq x_2 \leq 10, \end{cases} \end{cases}$$

where we have introduced other lower and upper bounds on the design variables than before, and where we take $C_1 = C_2 = 1$. We will solve this problem by using MMA. The asymptotes are updated during the iterations according to the rules described on page 68.

The script `ex3.m` is the main program file to solve $(\mathbb{P})_1$ using MMA. In this script the lower and upper bounds of the design variables, the starting point, and the MMA parameters are given, as well as an implementation of the objective function, the constraint function and their derivatives. Run the program by typing `ex3` at the MATLAB prompt. At each iteration, the procedure is paused (press any key to continue to the next iteration) and the current solution and the MMA approximation of both the objective (g_0) and the constraint (g_1) are plotted. The exact objective and constraint functions are also plotted. The starting point has been put to $(x_1, x_2) = (5, 5)$ and $s_{\text{init}} = 0.1$. As you can see the iterates converge to the point obtained analytically in section 2.4.

1. Now make the MMA approximation less conservative by putting $s_{\text{init}} = 0.3$. What happens? Keep $s_{\text{init}} = 0.3$, but make MMA more conservative by changing s_{decr} from 0.7 to 0.5, and s_{incr} from 1.2 to 1.1. What happens? Run the function `plotmmahist(history)`, where input argument `history` is a structure array automatically generated by the script `ex3.m`. In MATLAB figure 2, the iteration histories of x_1 , L_1 , and U_1 are plotted (you may click and drag the legend box if it obstructs the curves). Figure 3 shows the same for the second design variable, whereas figure 4 and figure 5 shows the iteration histories of g_0 and g_1 . Keep $s_{\text{decr}} = 0.5$ and $s_{\text{incr}} = 1.1$, but put $s_{\text{init}} = 0.1$ again. Run `plotmmahist(history)` and note the difference in appearance of the plots.

2. By setting the lower and upper asymptotes to 0 and ∞ , respectively, a CONLIN approximation is obtained (why?). Does CONLIN manage to solve the problem? (Use Ctrl-C to terminate the MATLAB script.) Verify by plotting the MMA history. Note that this time the upper asymptotes U_j^k are not plotted.
3. For a general optimization problem it may be very difficult to find a starting point that is feasible, i.e. a point satisfying the constraints $g_i(\mathbf{x}) \leq 0, i = 1, \dots, I$. Therefore, it is important that solution algorithms are able to find (local) optima even if started outside the feasible set. Use CONLIN again, but modify the starting point to $(x_1, x_2) = (1, 1)$, which is outside the feasible set. What happens? Switch back to the “ordinary” MMA solver again (by setting `low=[]` and `upp=[]`) and use $s_{\text{init}} = 0.1$, $s_{\text{decr}} = 0.7$ and $s_{\text{incr}} = 1.2$. What happens?
4. The volume of the four-bar truss in example 4.2 should be minimized under the constraint that the elongation of bar 1, $\delta \leq \delta_0$. This results in the nonconvex optimization problem $(\mathbb{P})_2$:

$$(\mathbb{P})_2 \quad \begin{cases} \min_{x_1, x_2} & g_0(x_1, x_2) = x_1 + x_2 \\ \text{s.t.} & \begin{cases} g_1(x_1, x_2) = \frac{8}{16x_1 + 9x_2} - \frac{4.5}{9x_1 + 16x_2} - 0.1 \leq 0 \\ 0.2 \leq x_1 \leq 2.5, \quad 0.2 \leq x_2 \leq 2.5. \end{cases} \end{cases}$$

This is the same problem as the problem on page 64. We will solve it numerically by generating a sequence of convex subproblems.

5. The script `ex2a.m` is used to solve $(\mathbb{P})_2$. Solve the problem by using the starting point $(x_1, x_2) = (2, 1.5)$, and the MMA parameters $s_{\text{init}} = 0.5$, $s_{\text{decr}} = 0.7$ and $s_{\text{incr}} = 1.2$. What is the solution? What is the optimum objective value? Also try to use the starting point $(x_1, x_2) = (1.3, 2.4)$.
6. Set the lower and upper asymptotes to $-\infty$ and ∞ , respectively, to simulate SLP (why?). Use the two starting points in task 5. Does it work?
7. Solve the problem by CONLIN using the two starting points in task 5. What happens?
8. Let us assume that x_1 is *fixed* at $x_1 = 1.35$, and that only x_2 may be varied in the optimization. For this case, the script `ex2b.m` is used. Use the same MMA parameters as in task 5. Choose a number of different starting points: $x_2 = 2.45, 1.5, 1.4, 1.3, 0.5$. What happens? The behavior is typical for nonconvex problems where there exist several local minima. What is the global minimum of this problem? What are the optimal objective values obtained by MMA? Are they smaller or greater than the one obtained in task 5? Why?